

Anil Seth



Exploring Big Data on a Desktop: Spark and HBase

The last issue of *OSFY* carried the column 'Exploring Big Data', which took a look at Apache Spark. This article explores HBase, the Hadoop database, which is a distributed, scalable big data store. The integration of Spark with HBase is also covered.

Spark can work with multiple formats, including HBase tables. Unfortunately, I could not get the HBase Python examples included with Spark to work. Hence, you may need to experiment with Scala and Spark instead. The quick-start documentation with Scala code is fairly easy to read and understand even if one knows only Python and not Scala (<http://spark.apache.org/docs/1.2.1/quick-start.html>). You can use the Scala example 'HBaseTest.scala' as the basis for further exploration.

Start the HBase server. In the Spark installation directory, you will need to start the Spark shell including HBase jars in the driver classpath. For example:

```
$ ./hbase-0.98.9-hadoop2/bin/start-hbase.sh
$ HBASE_PATH=../hbase-0.98.9-hadoop2/bin/hbase classpath
$ bin/spark-shell --driver-class-path $HBASE_PATH
scala>
```

You are now ready to enter Scala code in the interactive shell.

```
scala> import org.apache.hadoop.hbase.HBaseConfiguration
scala> val conf = HBaseConfiguration.create()
scala> val tableName="spark_table"
scala> import org.apache.hadoop.hbase.mapreduce.
TableInputFormat
scala> conf.set(TableInputFormat.INPUT_TABLE, tableName)
scala> import org.apache.hadoop.hbase.client.HBaseAdmin
scala> val admin = new HBaseAdmin(conf)
scala> admin.isTableAvailable(tableName)
res2: Boolean = false
```

So far, you have set up a connection to HBase and confirmed that the table you wish to create does not exist.

```
scala> import org.apache.hadoop.hbase.{HTableDescriptor,
HColumnDescriptor}
scala> val tableDesc = new HTableDescriptor(tableName)
scala> tableDesc.addFamily(new HColumnDescriptor("cf").
```

```
getBytes())
scala> admin.createTable(tableDesc)
```

You have now created a new table, 'spark_table', with a column family, 'cf'. In another terminal window, in the HBase installation directory, you can run the HBase shell and verify that the table has indeed been created, and that it is empty.

```
$ bin/hbase shell
hbase(main):001:0> list
spark_table
=> ["spark_table"]
hbase(main):002:0> scan "spark_table"
ROW                                COLUMN+CELL
0 row(s) in 2.5640 seconds
```

You can go back to the Scala shell prompt and enter some data.

```
scala> import org.apache.hadoop.hbase.client.{HTable, Put}
scala> val table = new HTable(conf, tableName)
scala> var row = new Put("dummy").getBytes()
scala> row.add("cf".getBytes(), "content".getBytes(), "Test
data".getBytes())
scala> table.put(row)
scala> table.flushCommits()
```

You have now created a connection to the HBase table and created a row with the key 'dummy' and the content 'Test data', in a column, 'cf:content'. You can verify that this data has indeed been stored from the HBase shell, as follows:

```
hbase(main):003:0> scan "spark_table"
ROW                                COLUMN+CELL
dummy                             column=cf:content,
                                timestamp=1427880756119, value=Test
data 1 row(s) in 0.1200 seconds
```